

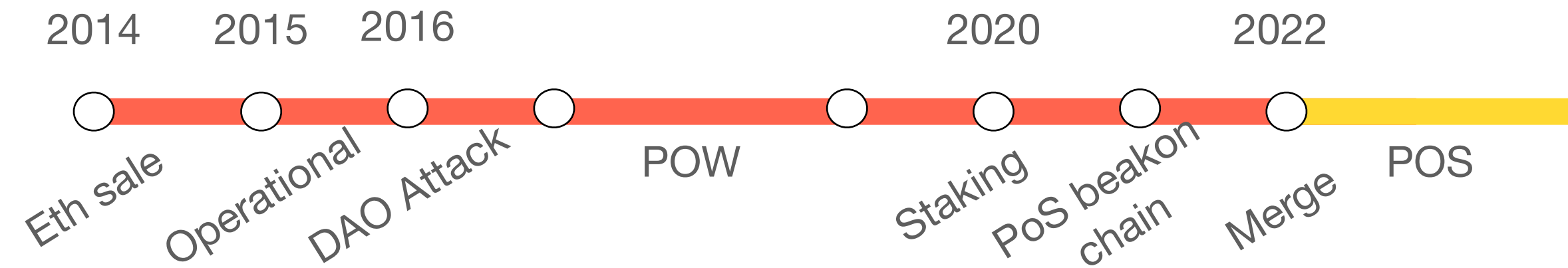
Ethereum

Introduction

Leander Jehl

Ethereum

Timeline



- 2014 Initial Ether sold on bitcoin
- 2016 DAO Attack: A Hacker exploited a smart contract bug, the community decided to undo attack, by a special state transition
- 2020 Possible to stake ether and participate in PoS beakon chain, that votes on blocks created by PoW
- 2022 PoW depricated and Ethereum using PoS to create bocks

From PoW to PoS

Ethereum - PoW

- Fast blocks (every 12sec)
- Structured P2P network
- Uses PoW hashing function not suited for ASICs
- Uncles: Blocks lost in a fork still get some reward

Ethereum - PoS

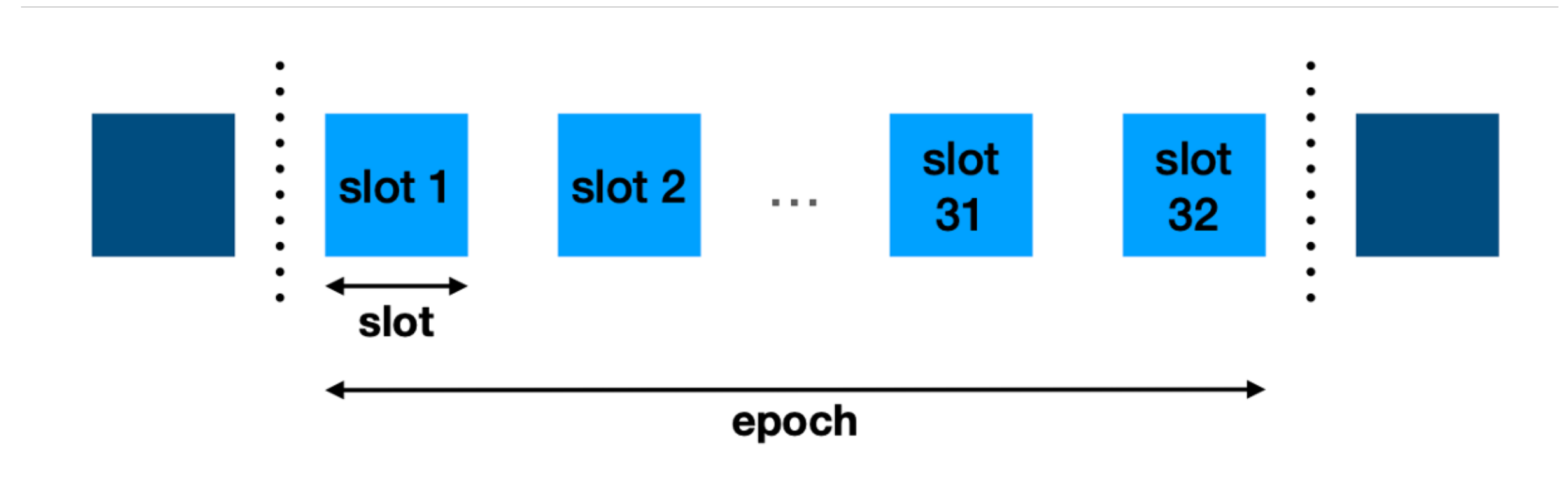
- One staker (validator) needs to deposit 32 ether (>75.000\$ 2024)
- Stake can be slashed, but not increased
-> 1.000.000 validators
- Validators need to run a node
- Can run one node for multiple validator ids.

Problem: Collecting signatures from
1mill nodes takes a lot of time.

Ethereum - PoS

Slots, Epoch, and Time

- One epoch is 6.4 minutes
- Each validator needs to vote once in each epoch (vote -> reward, no vote -> get punished)
- Epoch divided in 32 slots (each 12 sec)
- Validators divided in 32 committees
- Vote in your slot!

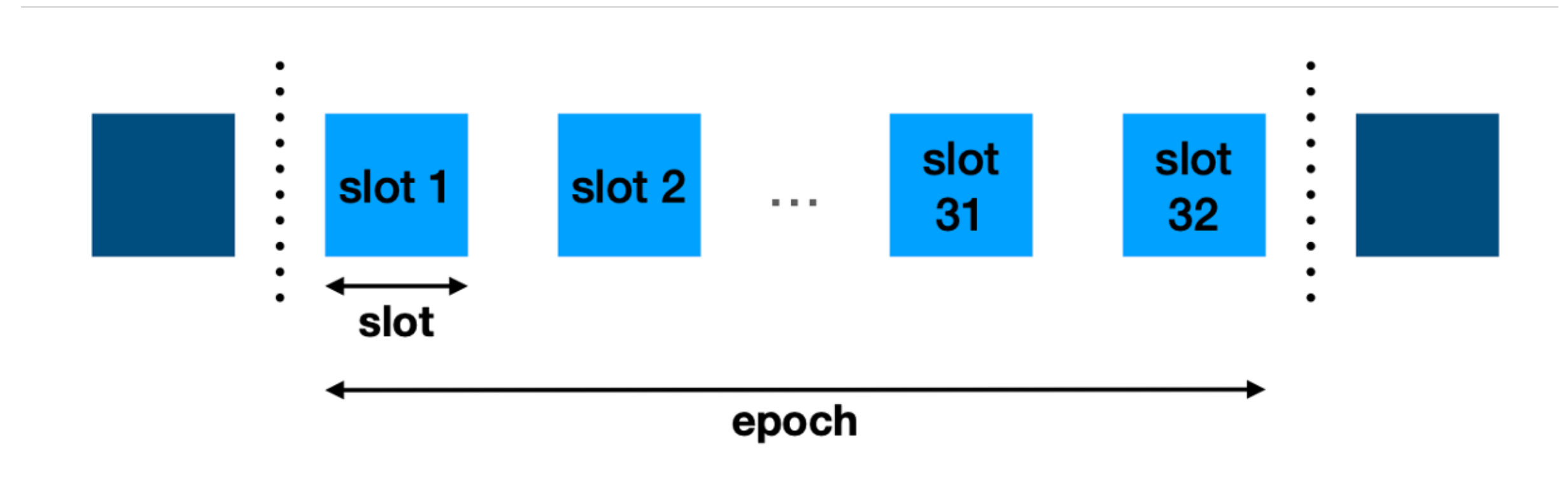


One block every 12 sec without
requiring one vote every 12 sec!

Ethereum - PoS

Proposers

- At the start of the epoch
32 random validators (proposers)
are selected to propose blocks
- Proposers also collect votes
- Proposers get extra reward



Ethereum - Forks

Two mechanisms:

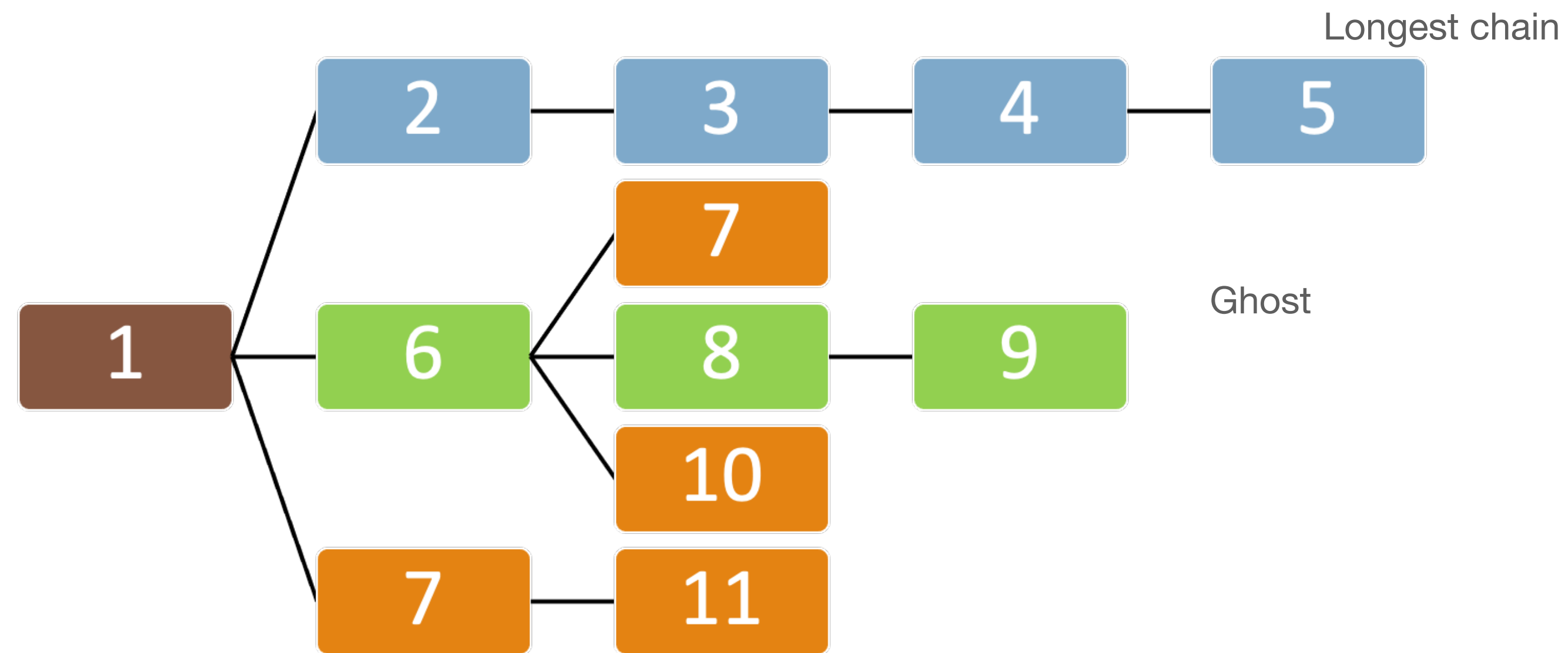
- **LMD Ghost:** Ensure new blocks can be created.
- **Casper FFG:** Create checkpoints
- Creating checkpoints is slow
- Forks can still arise between checkpoints

Ethereum - Forks

LMD Ghost

LMD Ghost is a fork choice rule (like longest chain):

- Rule which fork to extend



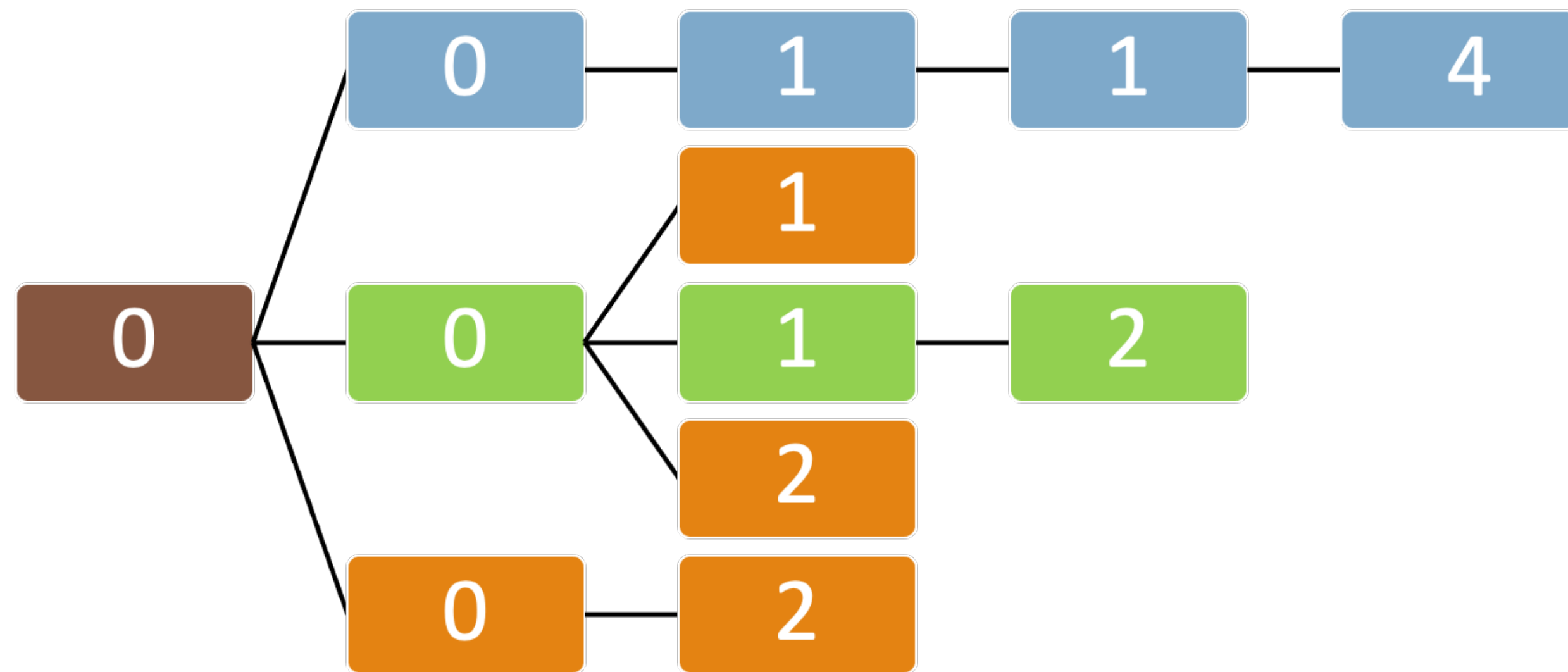
Example: GHOST - Greedy heaviest subtree

Ethereum - Forks

LMD Ghost

LMD Ghost: Extend fork with most votes

- Every validator can vote for one block



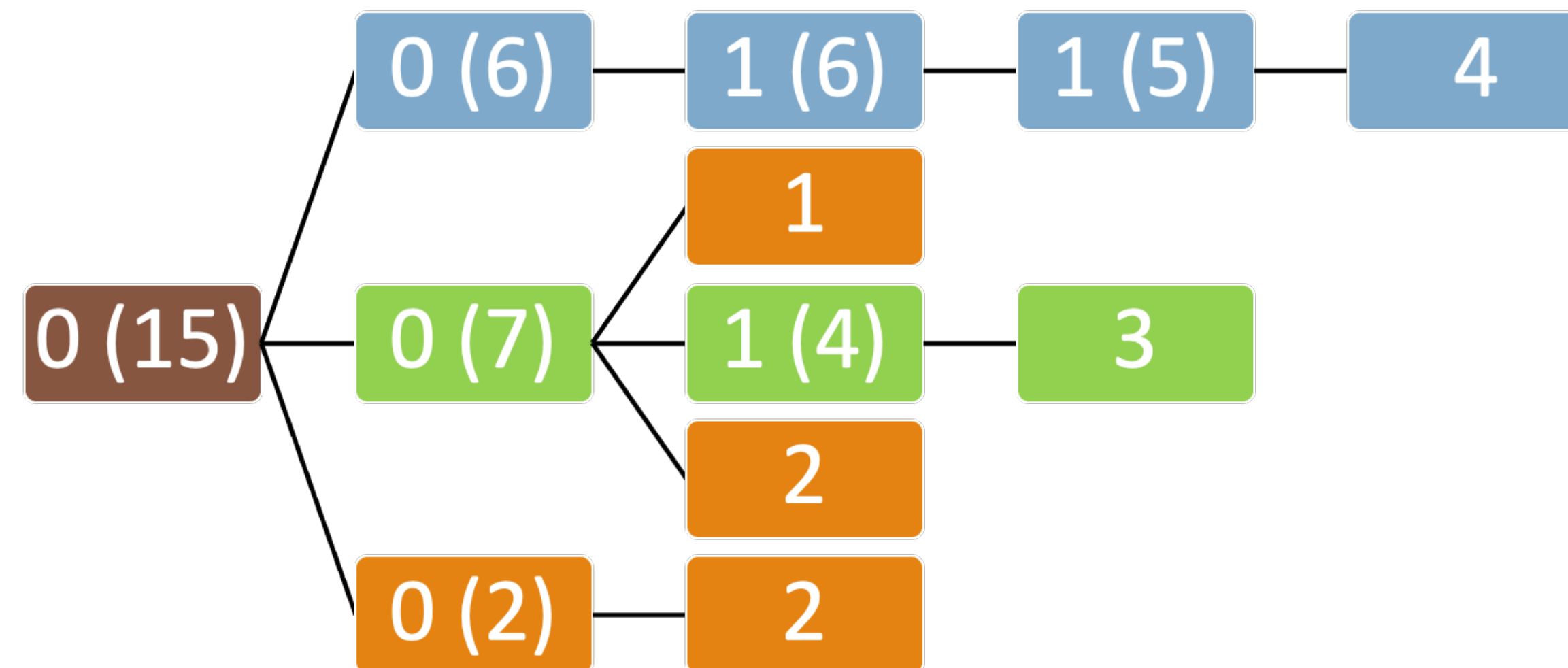
LMD Ghost example: Blocks with different count of votes

Ethereum - Forks

LMD Ghost

LMD Ghost: Extend fork with most votes

- Every validator can vote for one block
- Votes are summed for parents



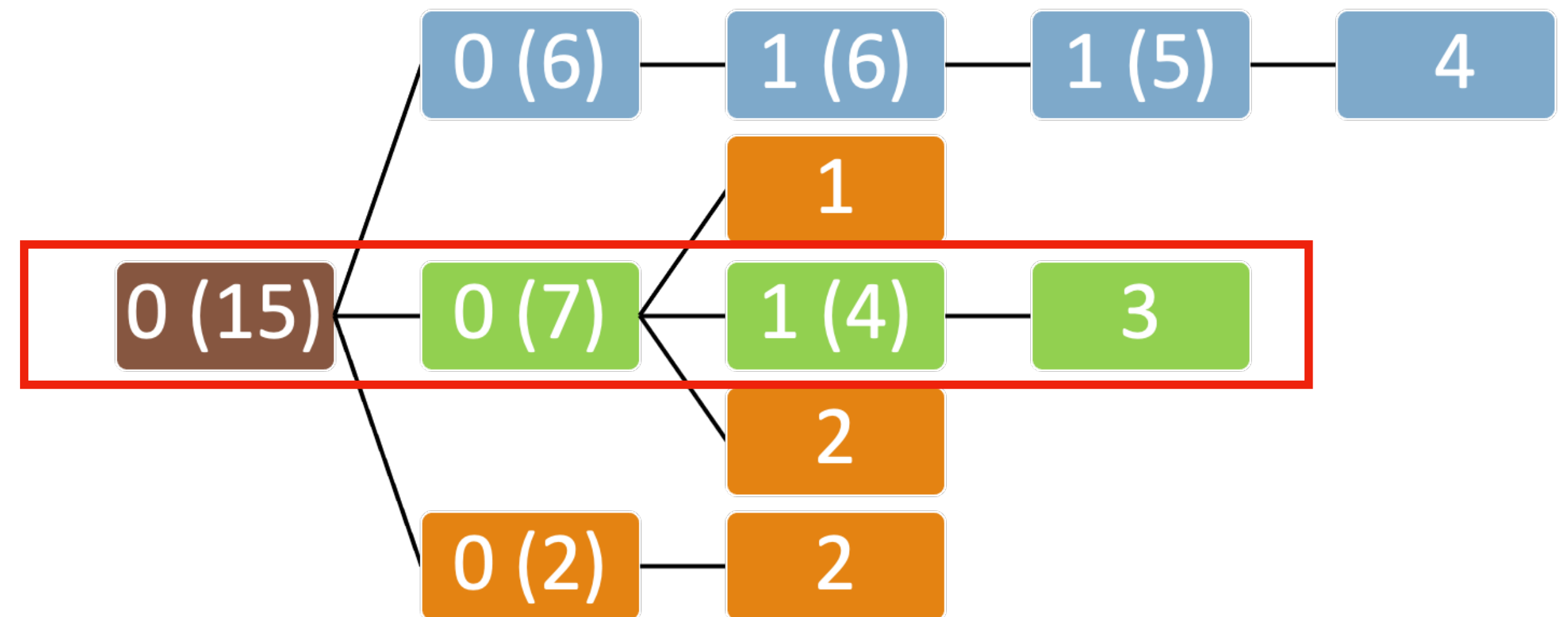
LMD Ghost example: Votes for children are summed up

Ethereum - Forks

LMD Ghost

LMD Ghost: Extend fork with most votes

- Every validator can vote for one block
- Votes are summed for parents
- Choose fork with most votes



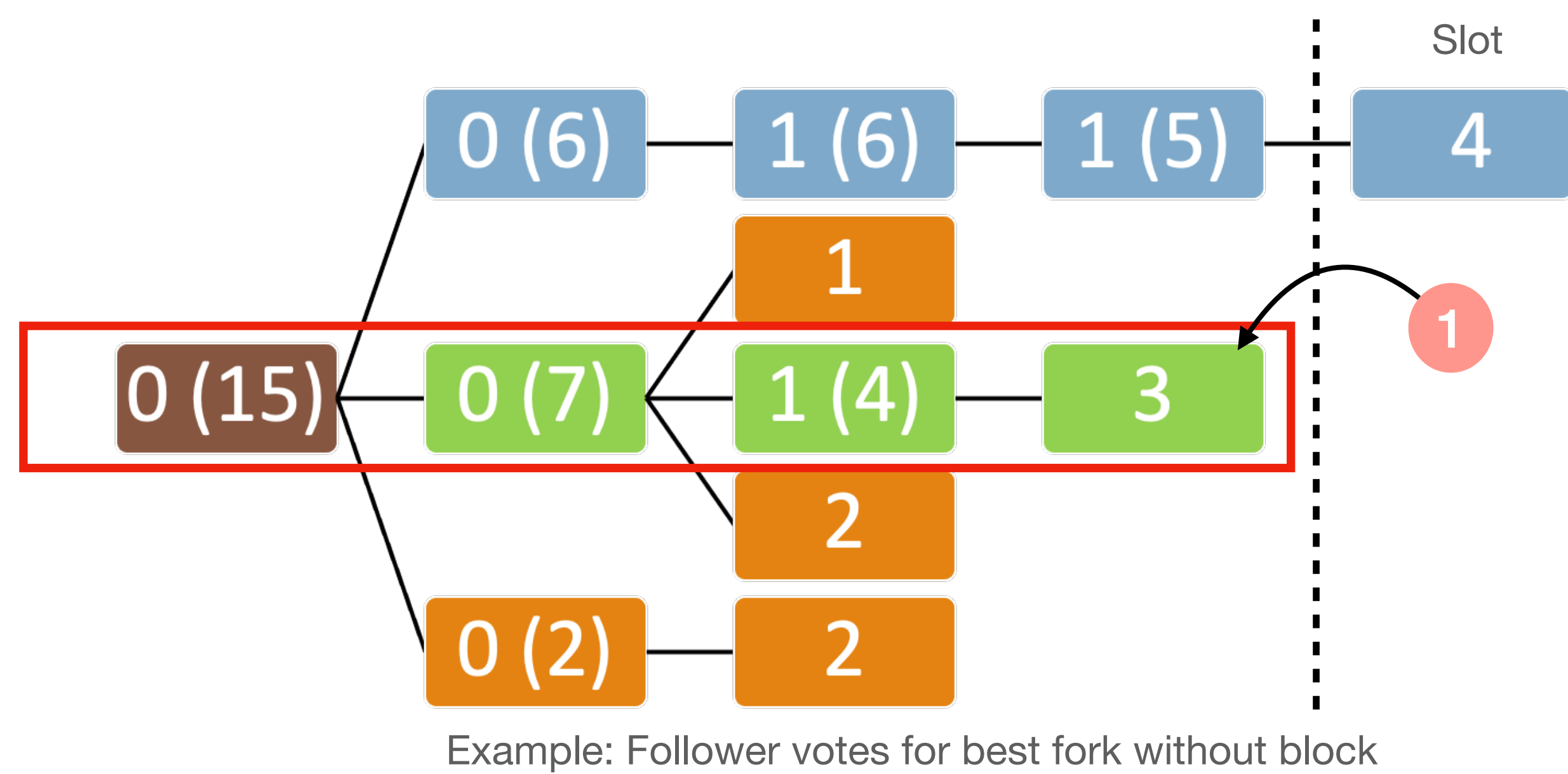
Ethereum - Forks

LMD Ghost

LMD Ghost: Extend & Vote for fork with most votes

Slot Leader: Create block extending best fork

Follows: Vote for best fork, even without block in current slot

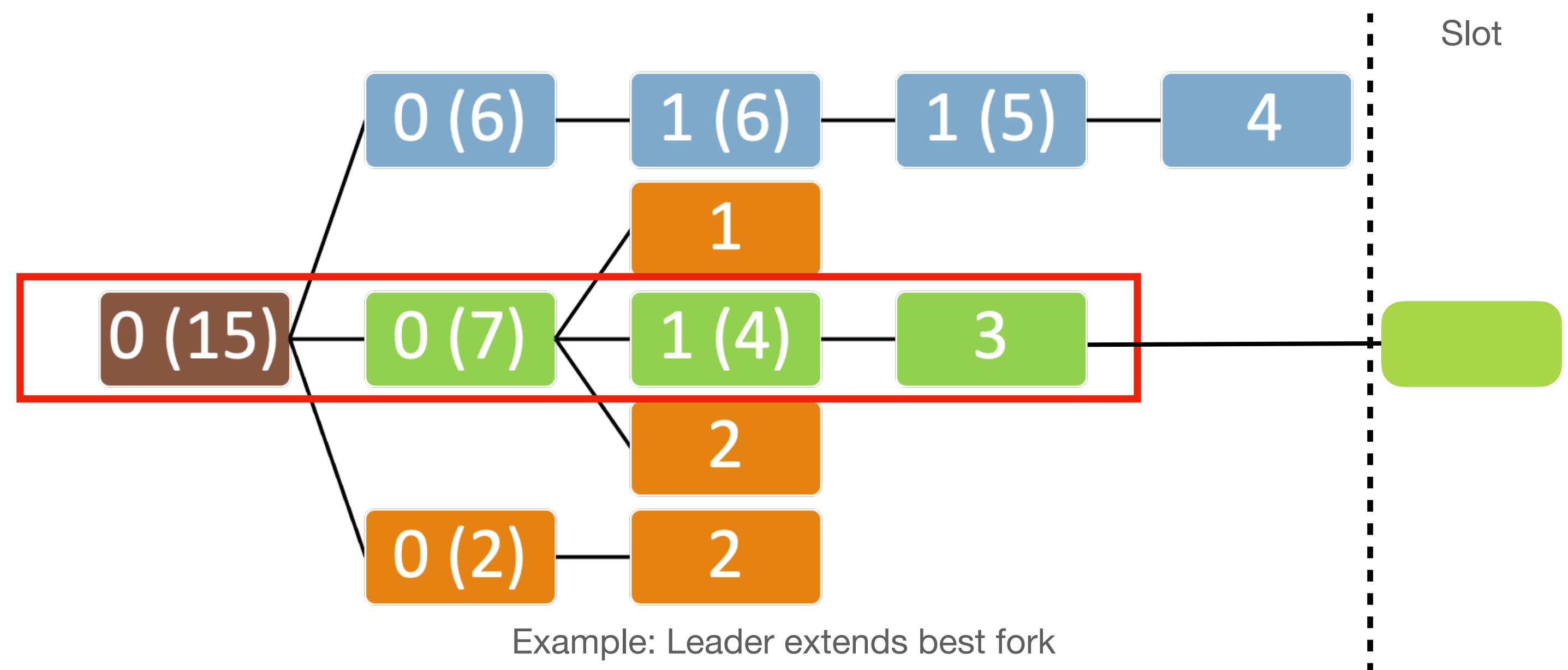


LMD Ghost

LMD Ghost: Extend & Vote for fork with most votes

Slot Leader: Create block extending best fork

Follows: Vote for best fork, even without block in current slot

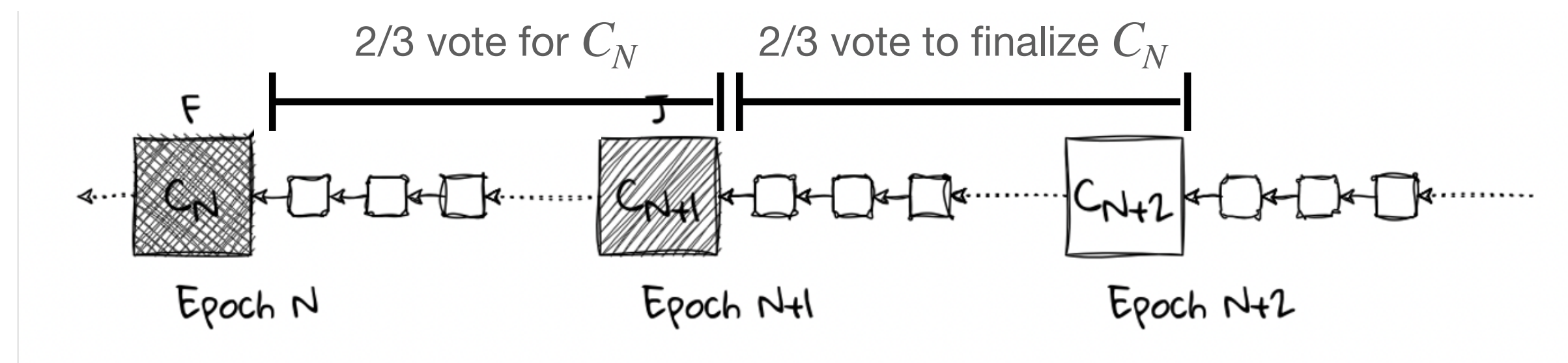


Ethereum - Forks

Casper FFG

Decide on checkpoint (consensus)

- Validators can vote to create checkpoint
- Need 2/3 of validators to vote for one checkpoint
- Needs to happen in 2 consecutive epochs

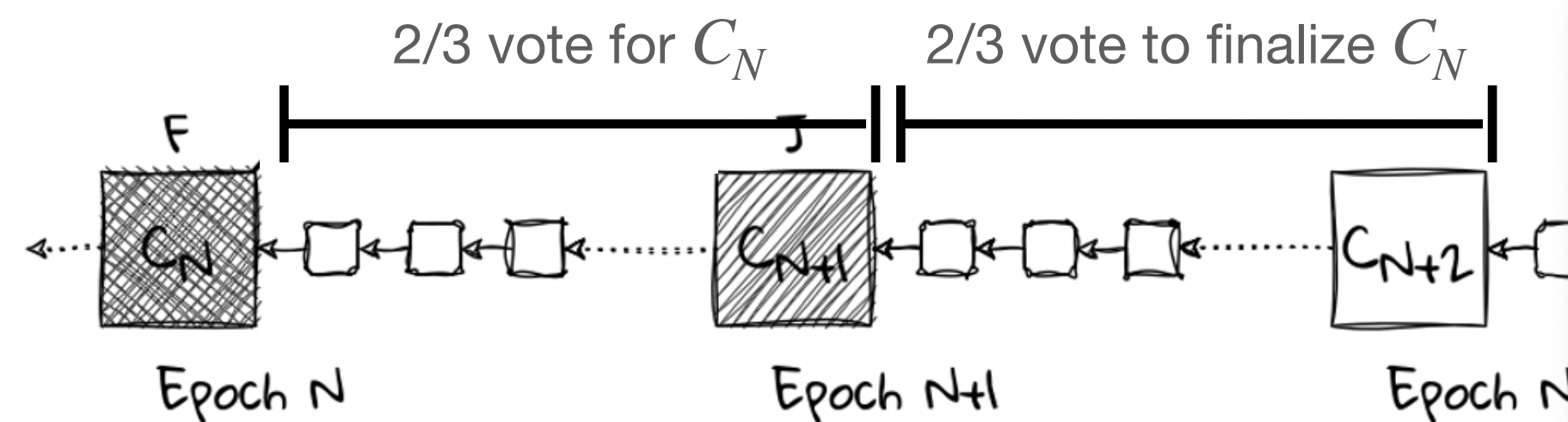


Ethereum - Forks

Casper FFG

Decide on checkpoint (consensus)

- Validators can vote to create checkpoint
- Need 2/3 of validators to vote for one checkpoint
- Needs to happen in 2 consecutive epochs



In fault free cases, creating a
checkpoint takes 2 epochs
12 minutes

Ethereum vs Bitcoin (consensus)

PoS

Throughput ~ 40tx/s

Confirmation: 20min

PoW

Throughput ~ 10tx/s

Confirmation 1h

What are Smart Contracts

Bitcoin scripts scripts

Spending conditions

Transactions:

$$tx = \langle \underbrace{[(id_1, \sigma_1), (id_2, \sigma_2)]}_{\text{Inputs}}, \underbrace{[(pk_a, value_a), (pk_b, value_b)]}_{\text{Outputs}} \rangle$$

Bitcoin scripts scripts

Spending conditions

Transactions:

$$tx = \langle \underbrace{[(id_1, rd_1), (id_2, rd_2)]}_{\text{Inputs}}, \underbrace{[(s_a, value_a), (s_b, value_b)]}_{\text{Outputs}} \rangle$$

- s_a a **spending condition**: output can be used if a value is supplied, that evaluates s_a to true
- rd_1 a **redeeming argument**: should ensure the script s_{id_1} returns true

UTXO scripts

Examples

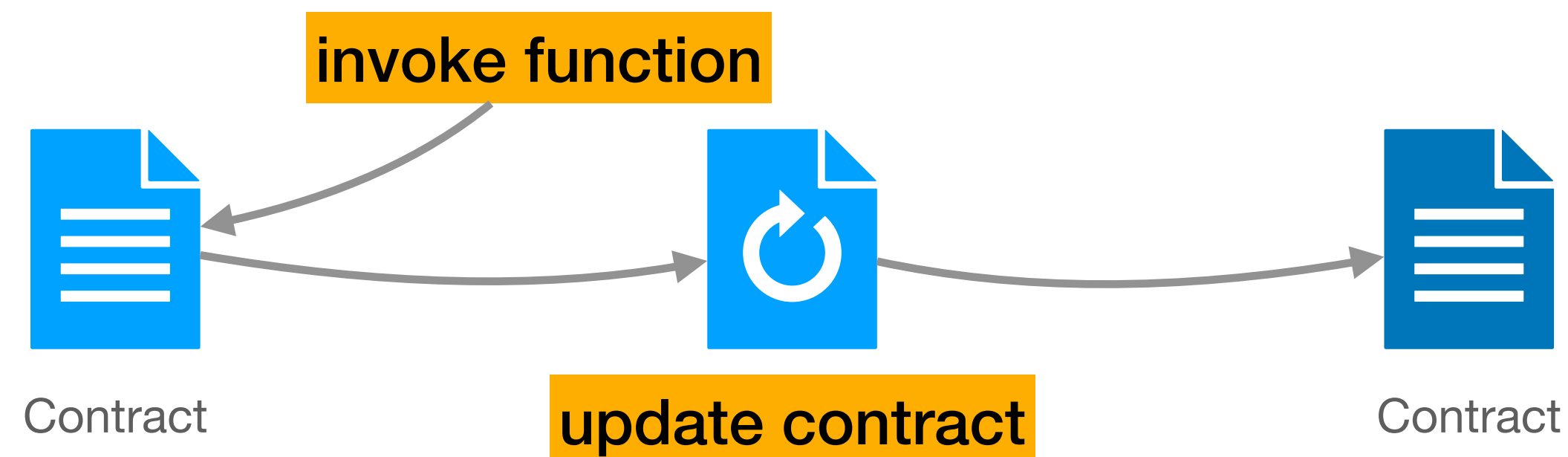
Name	Spending condition	Redeeming argument	
P2Pk Pay to public key	Public key	Signature	
P2PkH Pay to public key hash	Hash of Public key	Public key and signature	
Multisig	m public keys and parameter k	k signatures	

Ethereum

Smart Contracts

In ethereum, a contract is like an object from OOP, with fields and methods

- variables containing state (stored in account, mutable)
- functions



Ethereum

Example: Simple Storage

compiler version

```
pragma solidity ^0.5.11;
```

contract

```
contract SimpleStorage {
```

```
    uint256 public storedData;
```

state

```
    function get() public view returns (uint256){  
        return storedData;  
    }
```

```
    function set(uint x_) public {  
        storedData = x_;  
    }
```

functions

```
}
```

Ethereum

Example: Simple Storage

Simple online IDE: <https://remix.ethereum.org/>

Fun tutorial: <https://cryptozombies.io/>

- Constructors
- Basic types and collections
- Visibility (private, public)
- Inheritance
- Modifiers (view, pure)

Ethereum

Example: Simple Storage

Who can invoke functions?

Who can view values?

Who can change the code?

```
pragma solidity ^0.5.11;

contract SimpleStorage {
    uint256 public storedData;

    function get() public view returns (uint256){
        return storedData;
    }

    function set(uint x_) public {
        storedData = x_;
    }
}
```

Ethereum

Example: Simple Storage

Who can invoke functions?

- any user

Who can view values?

- anyone

Who can change the code?

- noone

```
pragma solidity ^0.5.11;

contract SimpleStorage {
    uint256 public storedData;

    function get() public view returns (uint256){
        return storedData;
    }

    function set(uint x_) public {
        storedData = x_;
    }
}
```

Ethereum

Smart Contract code

Smart contract code is immutable and public

- Anyone can trust smart contract (if it is not too complex)
 - No need to trust the creator of the contract
- No one can fix bugs in the contract
- Anyone can find and exploit bugs in the contract

Ethereum

Smart Contract code

- Assembly for Ethereum Virtual machine (EVM)
- Compiled from higher level language (Solidity)
- Stored in account (codeHash)

Ethereum

Smart Contract

Why use a smart contract

-

Ethereum

Smart Contract

Why use a smart contract

- No legal system to enforce paper contract
- Cheaper than paper contract
- Create a blockchain to do XYZ.

Ethereum

Smart Contract Example

Name registry

provides a mapping between name strings and addresses (pub key hash)

- Buy name
- Transfer name
- Lookup name

Ethereum

Smart Contract Example - Name registry

State and constructor

```
contract SimpleNameRegistry {  
    uint256 public constant PRICE = 1_000_000 gwei; // 0.001 ether  
  
    address public owner;  
    mapping(string => address) private nameOwner;  
  
    constructor() {  
        owner = msg.sender;  
    }  
}
```

Ethereum

Smart Contract Example - Name registry

Buy Name

```
function buyName(string calldata name) external payable {  
    require(msg.value >= PRICE, "Insufficient payment");  
    require(nameOwner[name] == address(0), "Name already taken");  
  
    nameOwner[name] = msg.sender;  
}
```

Vulnerable to front running!

Ethereum

Smart Contract Example - Name registry

Lookup Name

```
function lookupName(string calldata name) external view returns (address) {  
    return nameOwner[name];  
}
```

Needed for other contracts.

Ethereum

Smart Contract Example - Name registry

Transfer Name

```
function transferName(string calldata name, address to) external payable {  
    require(msg.value >= PRICE, "Insufficient payment");  
    require(nameOwner[name] == msg.sender, "Not the name owner");  
    require(to != address(0), "Invalid recipient");  
  
    nameOwner[name] = to;  
}
```

How to sell your name?

How does Ethereum enable Smart Contracts

Ethereum

Accounts

Ethereum uses accounts instead of UTXO.
Thus the state of Ethereum contains for every account:

- **address:** e.g. pub-key hash
- **balance:** amount of Ether the address owns
- **nonce:** sequence number of last transaction sent from this account
- **storage root:** *only for non-user accounts (contract account)*
- **code hash:** *only for non-user accounts (contract account)*



EOA
End-user-owned account

Ethereum

Accounts

Smart Contracts are also represented as accounts.

A contract account has:

- **address:** *e.g. hash from creator address & creation transaction nonce*
- **balance:** amount of Ether the address owns
- **nonce:** number of other contract created by this contract
- **storageRoot:** hash of data stored in this contract
- **codeHash:** hash of the code of this contract

Can create different contracts with
the same code.
(Like objects with the same type)

Ethereum

Accounts

In a contract written in Solidity, you can access:

- The address of the current contract:

```
address contractaddress = address(this);
```

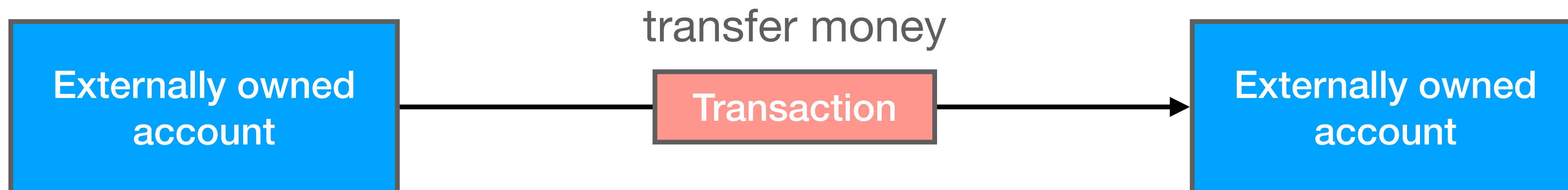
- The balance of the contract:

```
uint balance = contractaddress.balance;
```

Ethereum

Transactions and authentication

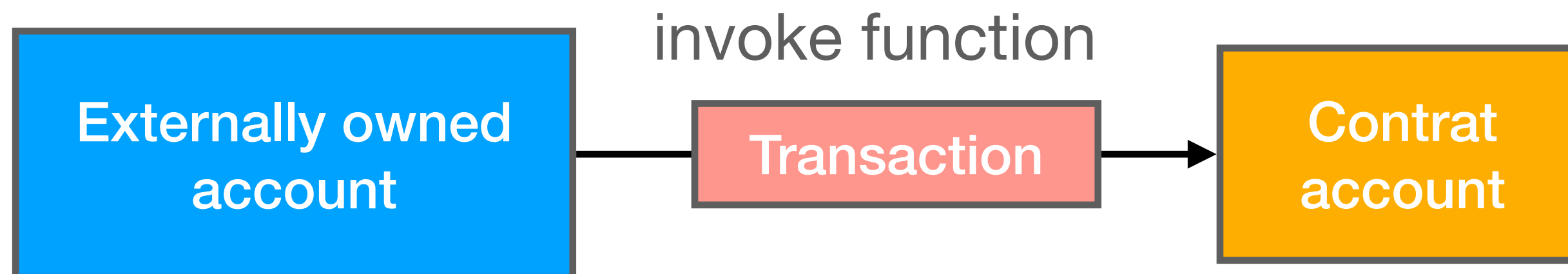
Transactions are used to transfer ether, invoke functions, and deploy new contracts.



Ethereum

Transactions and authentication

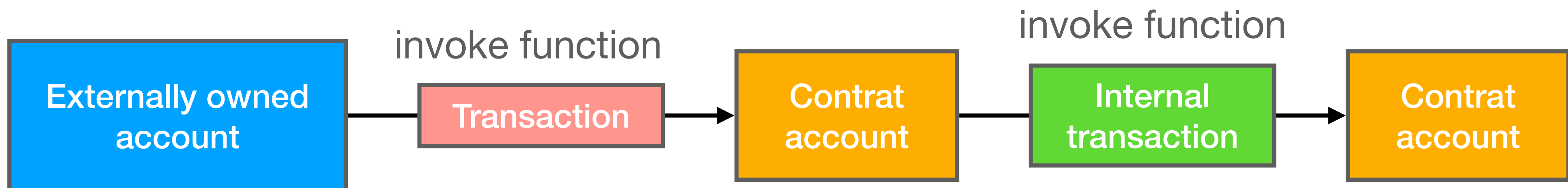
Transactions are used to transfer ether, invoke functions, and deploy new contracts.



Ethereum

Transactions and authentication

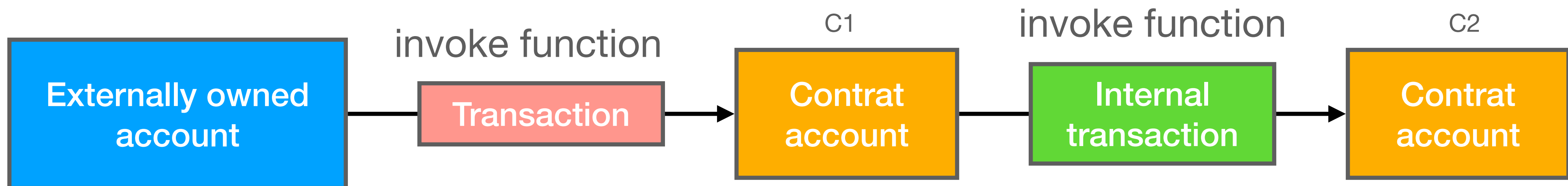
Transactions are used to transfer ether, invoke functions, and deploy new contracts.



Ethereum

Transactions and authentication

Transactions are used to transfer ether, invoke functions, and deploy new contracts.



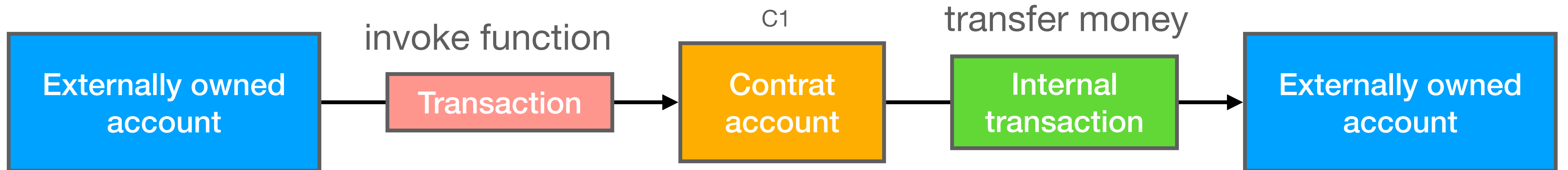
How does C1 know C2?

- C2 address supplied when deploying C1.
- C2 address supplied by transaction
- C2 address stored in contract state (can be updated)

Ethereum

Transactions and authentication

Transactions are used to transfer ether, invoke functions, and deploy new contracts.

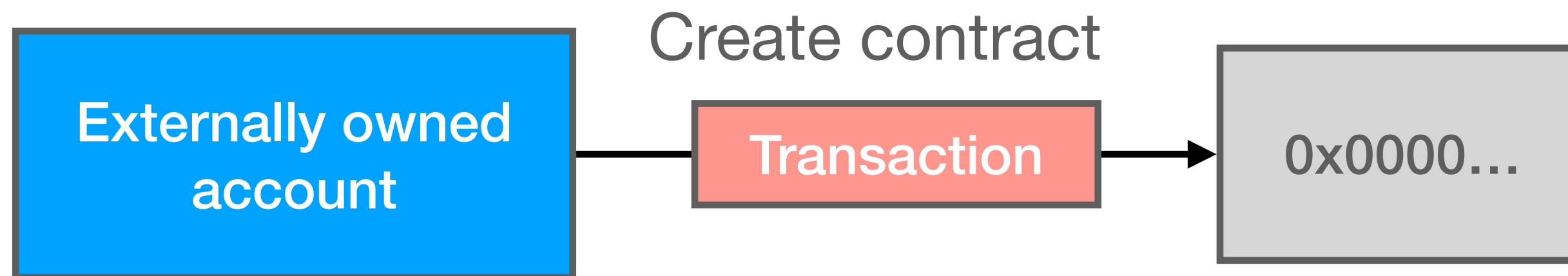


- Function can trigger transfer of money to user.

Ethereum

Transactions and authentication

Transactions are used to transfer ether, invoke functions, and deploy new contracts.

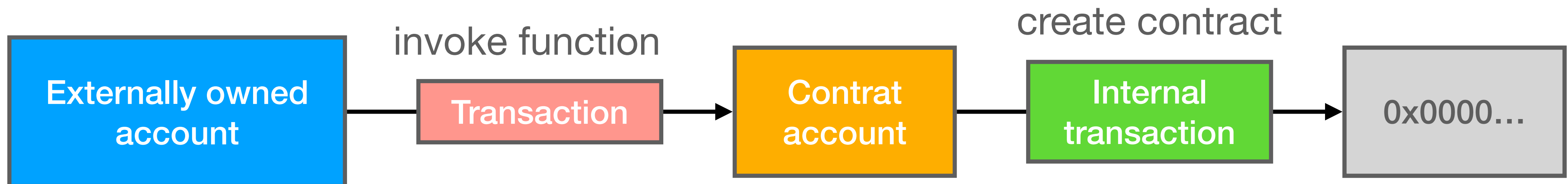


New contract byte code is sent in transaction.

Ethereum

Transactions and authentication

Transactions are used to transfer ether, invoke functions, and deploy new contracts.

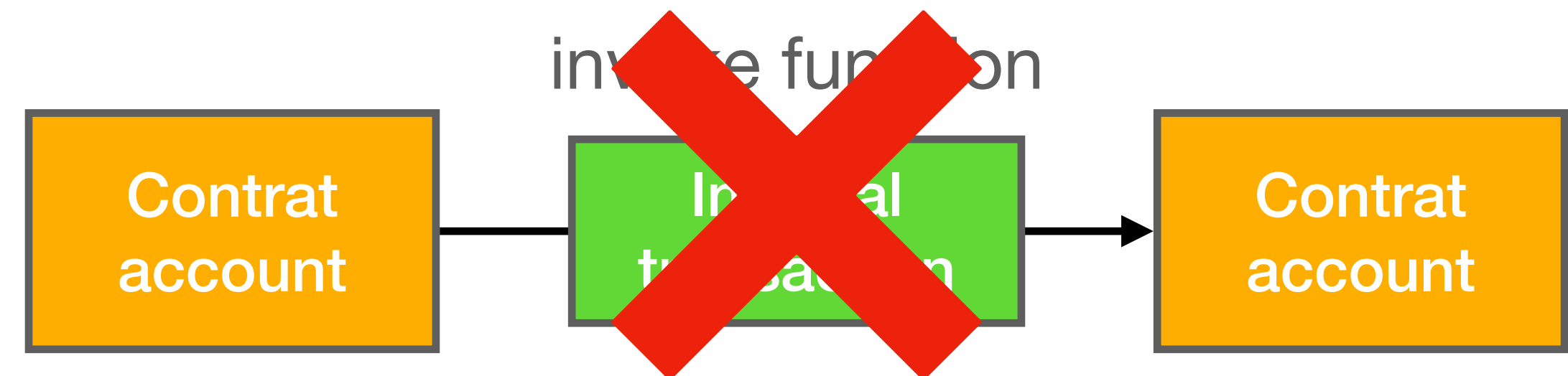


Ethereum

Transactions and authentication

Internal transactions cannot happen based on events.

Always need user to invoke.



Ethereum

Transactions and authentication

Transactions contain:

- *Nonce*: next sequence number for sender account
- *Gas price*: later
- *max gas*: later
- *Recipient*: destination Ethereum address
- *Value*: Amount of ether send to destination
- *Data*: Payload binary, e.g. function identifier and arguments
- *Signature*: Signature from sender, including his public key

Ethereum

Transaction validation

Transaction validation includes the following checks

- *Nonce*: is next sequence number for sender account
- Sender has sufficient balance to pay value and fees
- Transaction is correctly signed

When authenticating users in smart contract, we can rely on transaction validation!
Use *msg.sender* to access address invoking transaction.

Solidity example

```
contract SimpleBank {
    mapping(address => uint) private balances;
    address public owner;

    // function SimpleBank() deprecated syntax for
    constructor() public {
        owner = msg.sender;
    }

    function deposit() public payable returns(uint) {
        balances[msg.sender] += msg.value;
        return balances[msg.sender];
    }

    function withdraw(uint withdrawAmount) public returns (uint remainingBal){
        if (balances[msg.sender] >= withdrawAmount){
            balances[msg.sender] -= withdrawAmount;
            // this throws an error if fails.
            msg.sender.transfer(withdrawAmount);
        }
        return balances[msg.sender];
    }

    function balance() view public returns (uint) {
        return balances[msg.sender];
    }
}
```

Ethereum

Solidity example

What happens if data is empty?

- Money transfered to account. Default function run.

What is *msg.sender* for internal transactions?

- address of sending contract
 - a contract can have money in our bank!

Ethereum

Solidity exceptions

If a smart contract throws an exception, or error, state is reverted.

Ethereum

Solidity view functions

View functions are read only

- Do not require fees
- Read contract state at one node

How can you know that a read
from one node is correct?

**State stored in Ethereum
blockchain**

Bitcoin

Block structure

Header:

PrevBlockhash
Nonce
Timestamp

Transaction data

Merkle tree

Merkle tree allows to easily proof that a transaction is included in a block.

State of the blockchain (UTXO)
is not in the block.

Ethereum

Block structure

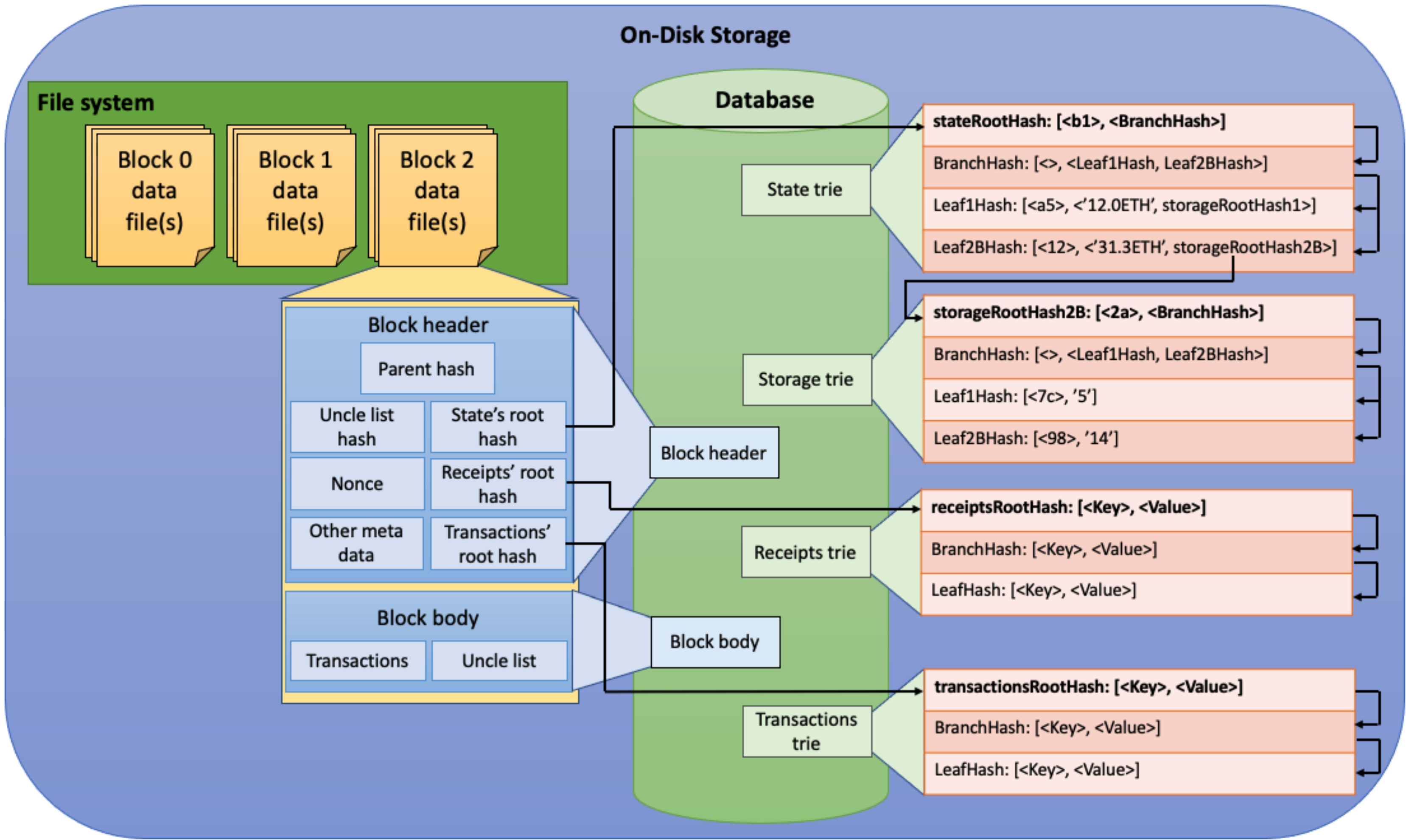
Header:

PrevBlockhash
Nonce
Timestamp

State root hash
Receipts root hash

Transaction data

Merkle tree



Ethereum

State trie

Stores accounts:

Account	
A	“ad1b...”
B	“ad92...”
C	“53e2...”

```
Address:
  [Value,
   Nonce,
   StorageRoot,
   CodeHash]
```

Trie:
Merkle tree that supports

```
update
lookup
proof
```

Ethereum

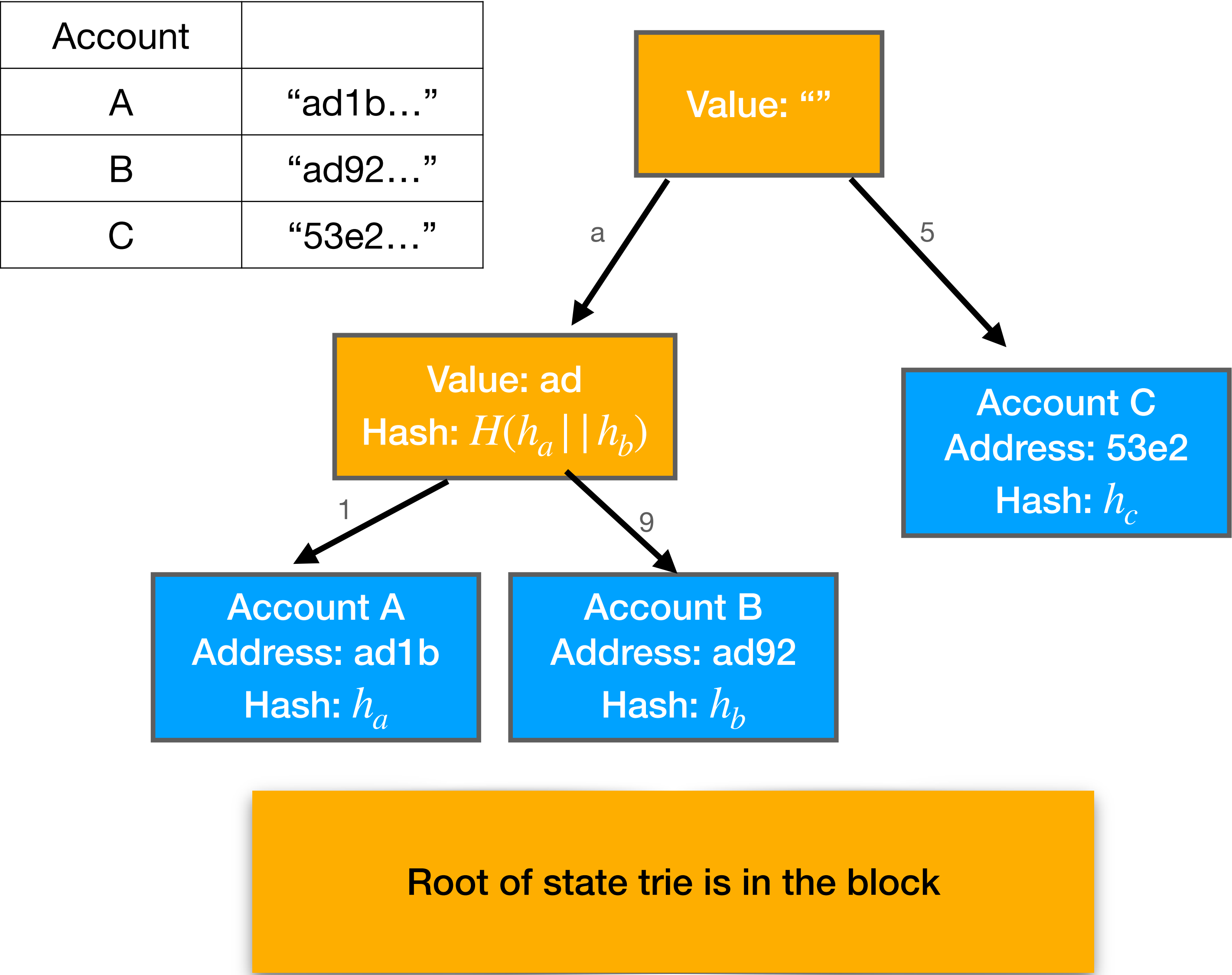
State trie

Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof



Ethereum

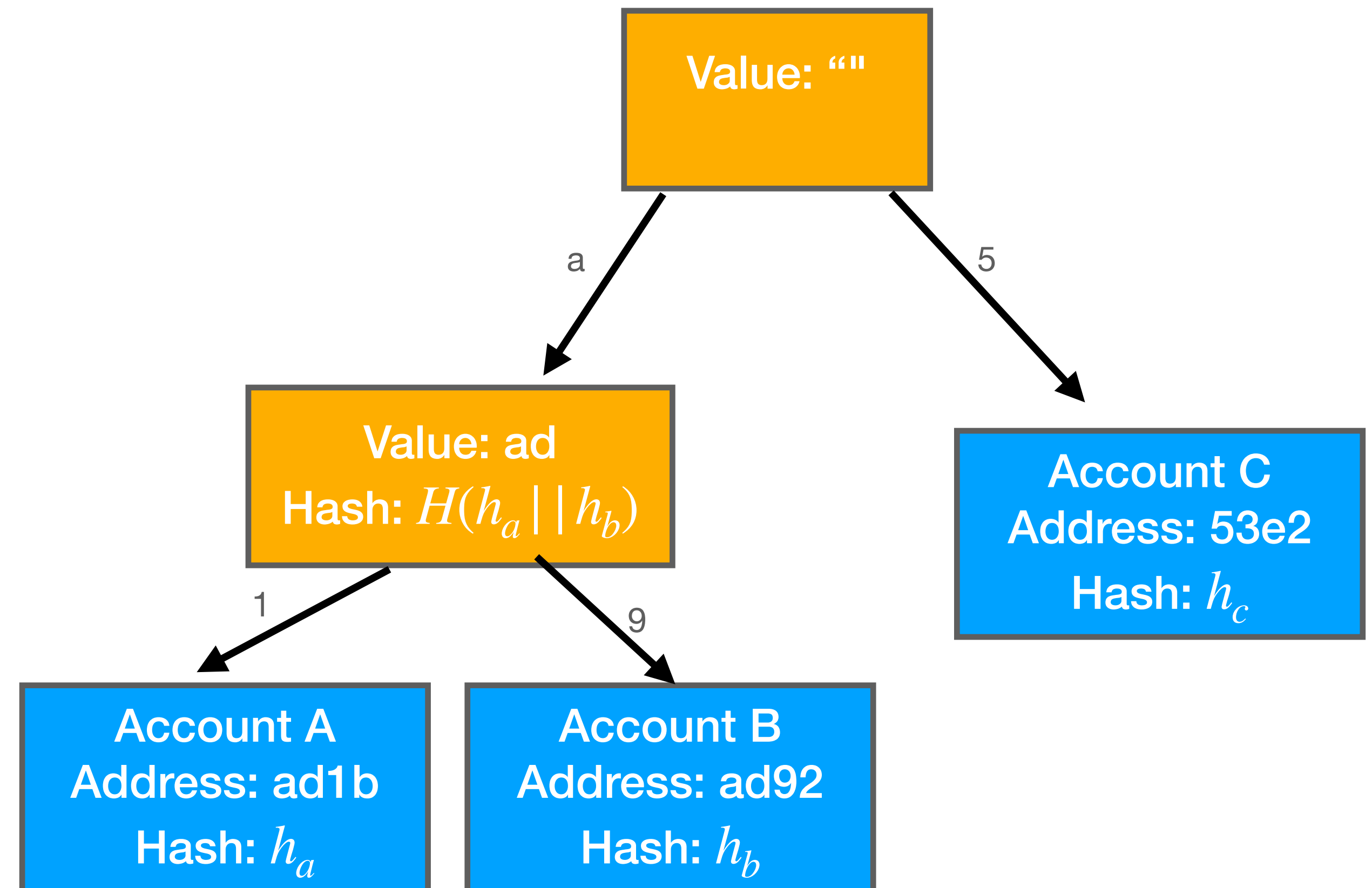
State trie

Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof



SC Variable changes

- > SC storage root changes
- > SC Account hash changes
- > State trie root changes

Ethereum

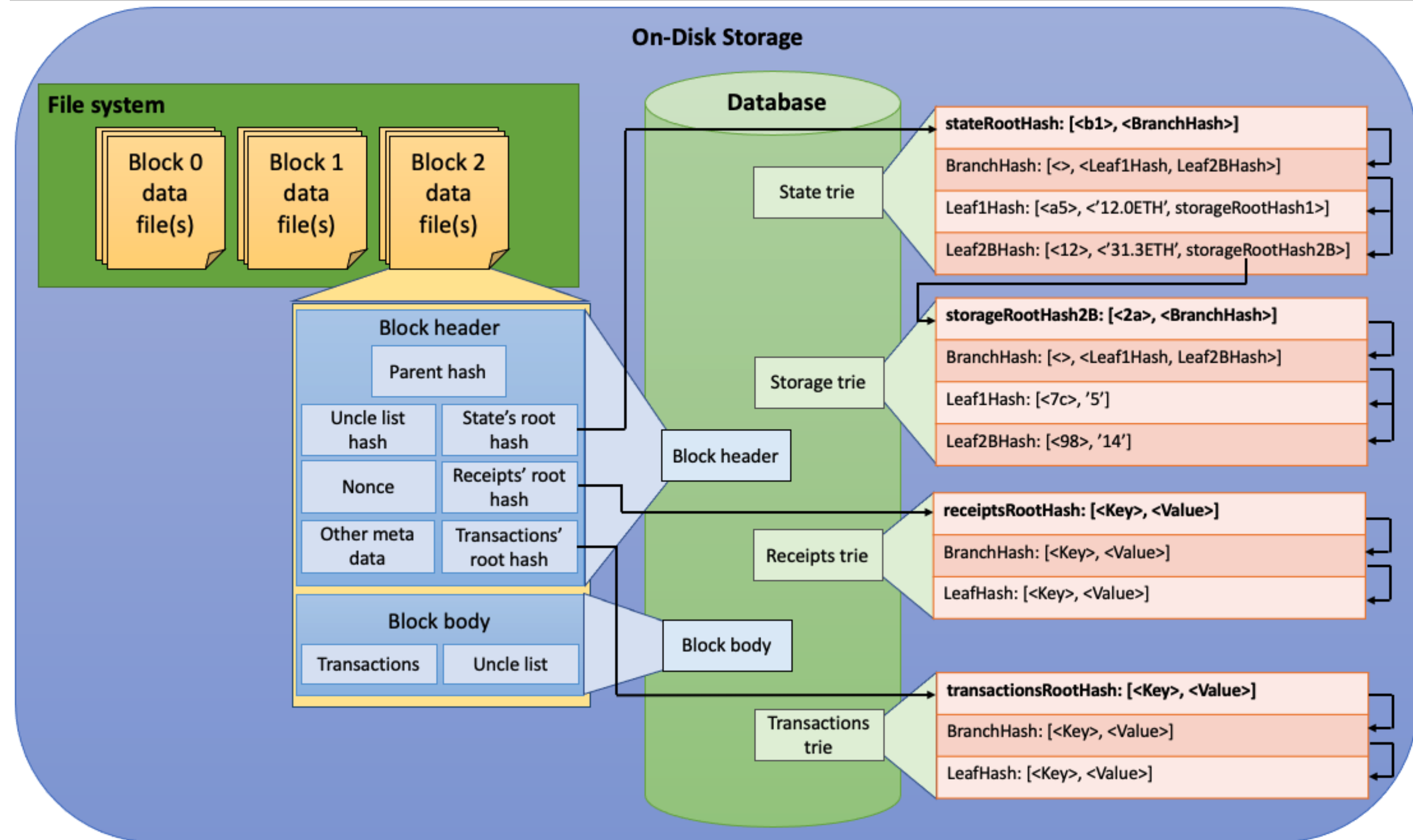
State trie

Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof



StorageRoot is the root of a different trie.

Ethereum

State trie

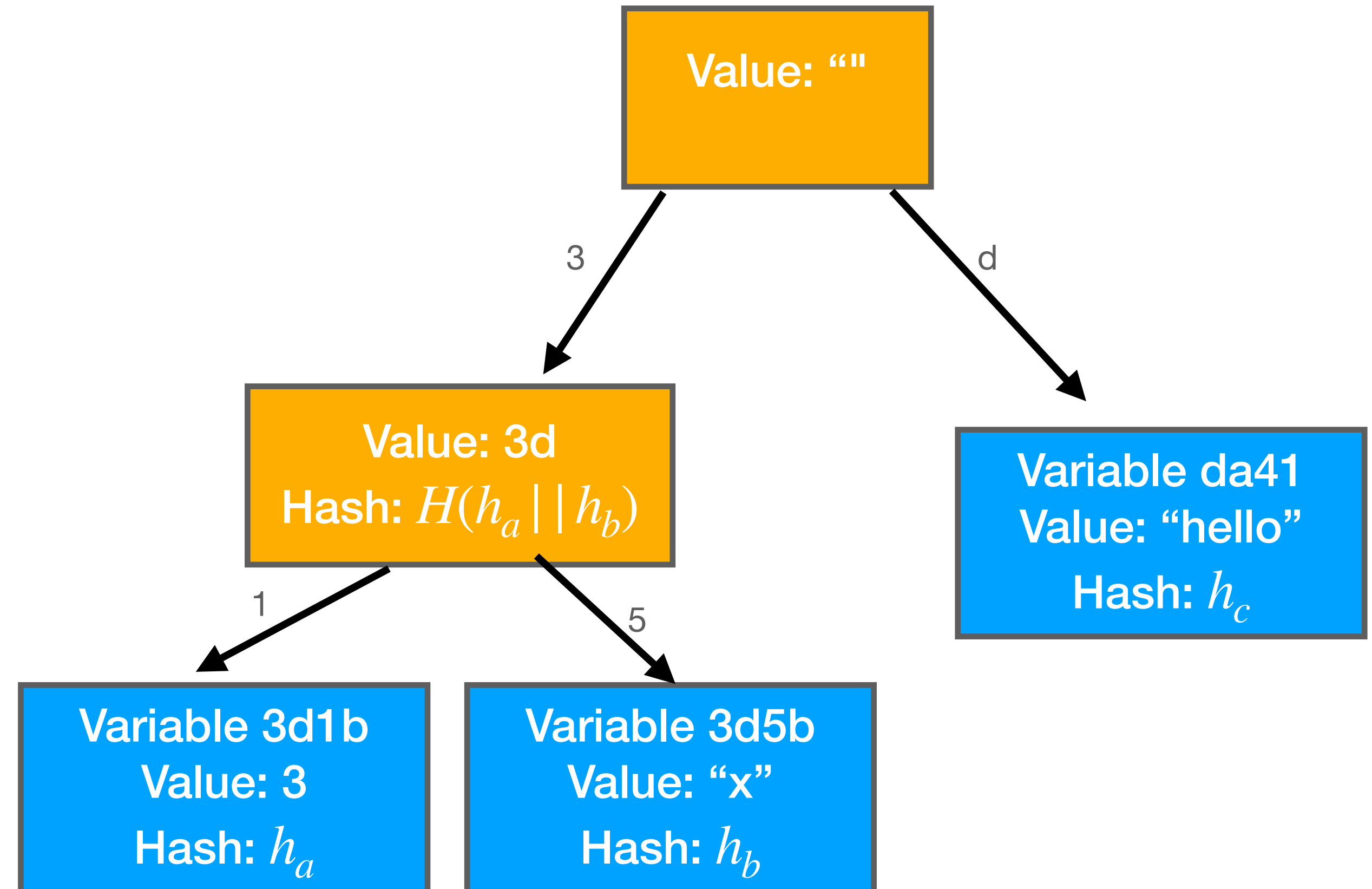
Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof

StorageRoot is the root of a different trie.



Ethereum

State trie

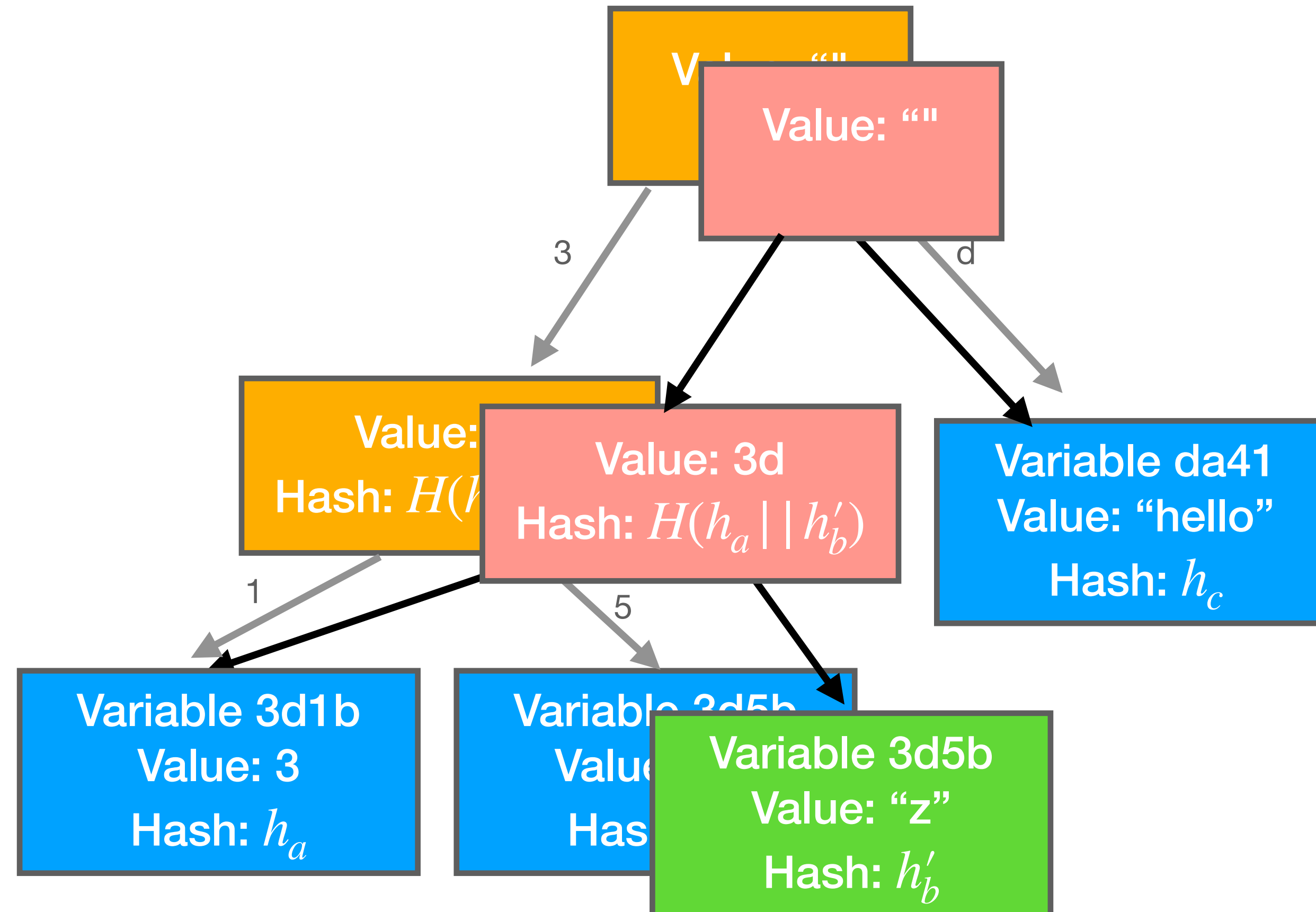
Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof

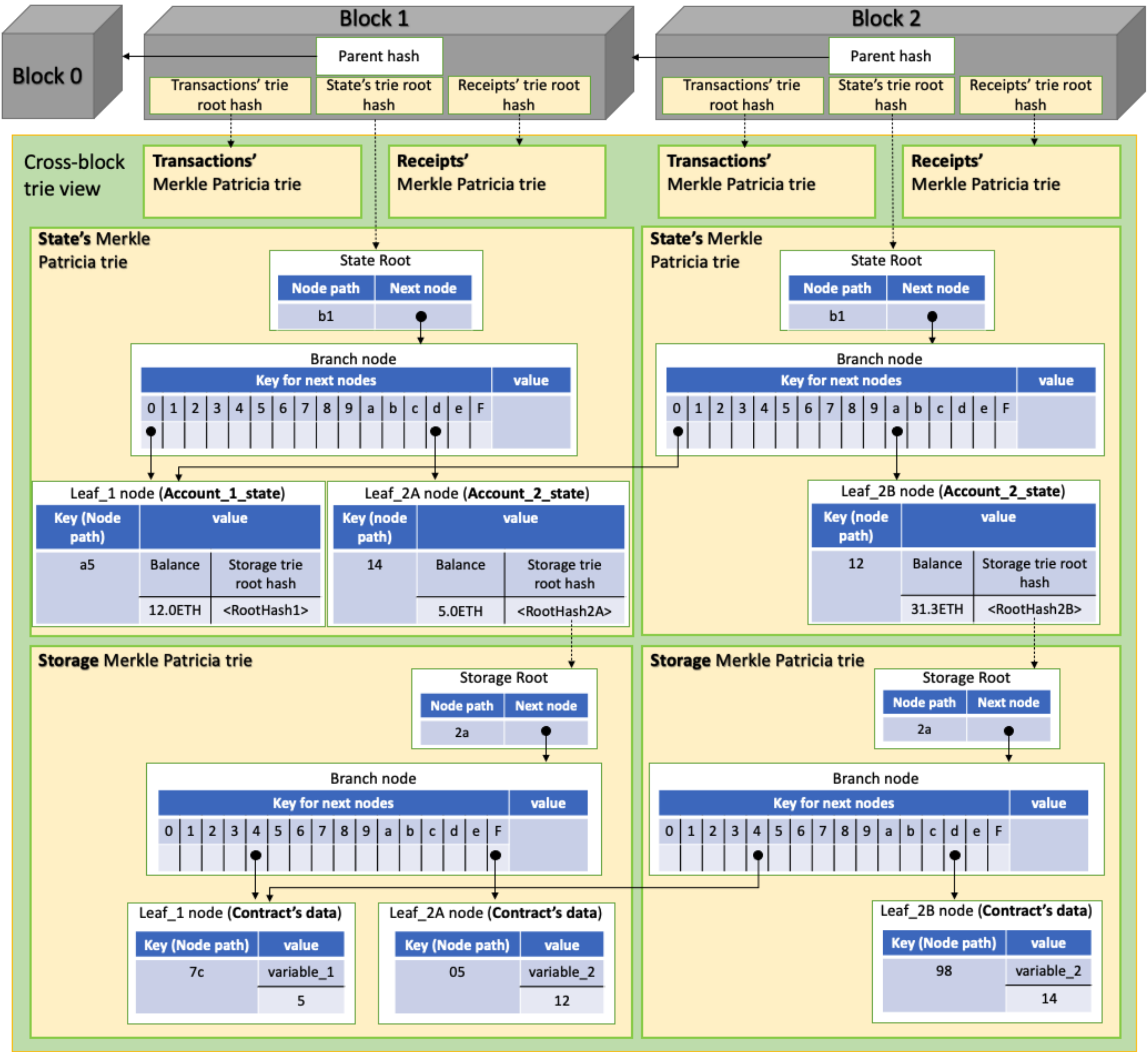
StorageRoot is the root of a different trie.



If a variable gets updated, new nodes get created on the path to root.

Ethereum

State trie



Ethereum

Read contract state

1. ask trusted node
2. receive inclusion proof for

stateRoot: storageTrie
account state: stateTrie

and block header

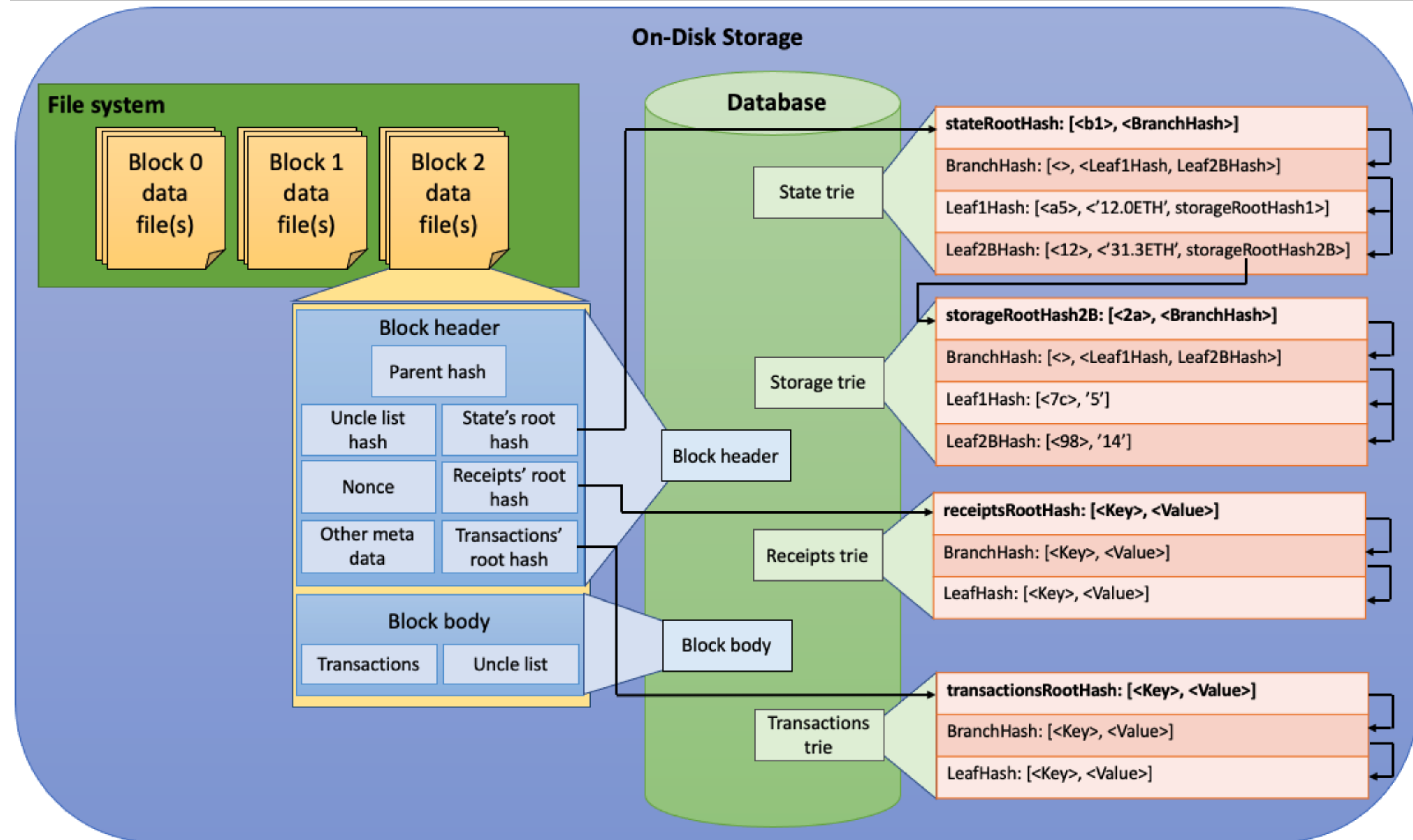
Ethereum

Receipts trie

Stores transaction results:

```
From: address
To: address
Status: ... // aborted?
Logs: events
ContractAddress address
// new contract address,
// if created
```

Return transaction results,
by emitting Events,
which are added to the logs.



Fees and gas

Ethereum

Gas

How to pay transaction fees in Ethereum?

- all bytecode instructions have a cost specified in Gas
- transaction has fixed cost in Gas
- especially: storing values is expensive

Why specific gas per instruction:

- An infinite loop will cost infinitely much gas -> avoid denial of service

Ethereum

Gas

Each transaction can have a different exchange rate *Gas to Ether*

- *Old way*: specify *gas price* and *gas limit*
- *New way*: specify *tip (priority fee)*, *max fee per gas* and *gas limit*

Old way specify *Gas price* and *Gas limit*

- *Gas price* is ether given per gas
- *Gas limit* is how much the transaction may spend at most

Ethereum

Gas

Each transaction can have a different exchange rate *Gas to Ether*

- *Old way*: specify *gas price* and *gas limit*
- *New way*: specify *tip (priority fee)*, *max fee per gas* and *gas limit*

New way specify *tip* and *max fee* (EIP-1559)

- *Gas price* is calculated $tip + base_fee$
- *Gas limit* is how much the transaction may spend at most

Ethereum

Gas

Gas Used	Gas Limit	Base Fee
28,593,288 (47.66%)	60,000,000	0.064 Gwei
24,485,633 (40.81%)	60,000,000	0.065 Gwei
46,074,987 (76.79%)	59,999,943	0.061 Gwei
24,909,707 (41.56%)	59,941,408	0.062 Gwei
27,580,444 (45.97%)	60,000,000	0.063 Gwei
54,381,329 (90.64%)	60,000,000	0.057 Gwei
27,364,250 (45.61%)	60,000,000	0.058 Gwei
23,418,256 (39.03%)	60,000,000	0.059 Gwei

Gas Used	Gas Limit	Base Fee
23,063,456 (38.48%)	59,941,408	0.163 Gwei
54,068,920 (90.11%)	60,000,000	0.148 Gwei
10,700,502 (17.83%)	60,000,000	0.161 Gwei
27,552,861 (45.92%)	59,999,943	0.163 Gwei
34,826,585 (58.10%)	59,941,408	0.159 Gwei
39,282,593 (65.47%)	60,000,000	0.153 Gwei
27,063,519 (45.11%)	60,000,000	0.155 Gwei
37,904,066 (63.17%)	60,000,000	0.15 Gwei

Gas price = tip💵👷 + base fee🔥

Ethereum

Gas

What happens if you hit the *Gas limit*?

- Exception is thrown and transaction reverted.
- Gas is still payed!

Which transactions are included?

- Miners will include transactions offering the highest gas price.
- Blocks have maximum amount of gas.